Darren Lee
danalee@ucsc.edu
4/7/2021

CSE 13S Spring 2021
Assignment 2: A Small Numerical Library
Design Document

New changes in red

Purpose

The purpose of this assignment is to create a small numerical library and test harness. The library will include arcSin, arcCos, arcTan, and Log functions. I will use trigonometric identities and Newton's method for arcSin and trigonometric identities alone for arcCos and arcTan. I will be using Newton's method to approximate the logarithm function. The test harness will allow me to compare my results to the standard <math.h> library and create a table showing the values and differences. ArcSin and arcCos will be tested in the range [−1,1] with steps of 0.1. ArcTan and Log will be tested in the range[1,10] with steps of 0.1.

Layout/Structure

- The code for the math library functions will be in mathlib.c. This will contain the arcSin, arcCos, arcTan, and Log functions.
- Mathlib-test.c contains the main() program and acts as a test harness for the math library.
- The getopt() command line options that mathlib-test.c supports are:
    - -a: to run all tests.
    - -s: to run arcsin tests.
    - -c: to run arccos tests.
    - -t: to run arctan tests.
    - -l: to run log tests.

Functionality

- The arcSin function will be implemented using Newton's method. Values close to -1 and 1 be calculated with Newton's method and trigonometric identities. I will use Newton's method and the identity $\cos(\arcsin(x)) = \sqrt{1-x^2}$ for 0<=x<=1 to find arcSin using arcCos(Sqrt(1-x^2)) for values close to 1. For values close to -1, I will use the guess - π to find an accurate arcSin value. Tests for the arcSin() function will use the range [-1,1] with steps of 0.1.

- The arcCos function will be implemented using the arcSin function. I will use the equation $\arccos(x) = \frac{\pi}{2} - \arcsin(x).$ Tests for the arcCos() function will use the range [-1,1] with steps of 0.1.
- The arcTan function will be implemented using the arcCos function and the equation

$$\arctan(x) = \arccos\left(\frac{1}{\sqrt{x^2+1}}\right), \quad x > 0.$$

Tests for the arcTan() function will use the range [1,10] with steps of 0.1.
- The Log function will be implemented using Newton's method by computing the inverse of e^x. The following equation will allow us to implement logarithms with base e:

$$x_{k+1} = x_k + \frac{y - e^{x_k}}{e^{x_k}}.$$

Log will only work for positive values because 0 and negative values are not in the domain of the natural log function. I used an assert macro to ensure that Log() only accepts positive inputs. Tests for the Log() function will use the range [1,10] with steps of 0.1.
- The tests for the math library accept the -asctl options. -a will run all tests, -s will run arcSin tests, -c will run arcCos tests, -t will run arcTan tests, -l will run Log tests. Entering in no options or entering an invalid option will print out program usage instructions.

Pseudocode for mathlib.c

```
Abs(x) function: (from Piazza)
        Return (x < 0) ? -x : x

Exp(x) function: (from Piazza)
        term = 1, sum = 1
        For (k=1; Abs(term) > EPSILON; k++):
                term *= x/k
                sum += term
        Return sum

Sqrt(x) function: (from Piazza)
        y = 1.0
        assert (x >= 0)
        For (guess = 0.0; Abs(y-guess) > EPSILON; y = (y + x / y) / 2.0):
                Guess = y
        Return y
```

Removed Pow function because it was unnecessary

arcSin(x) function:

```
        k = x
        prev = 0.0
        j = 1.0
        p = 0.0
        If (x > 0.99 or x < -0.99):
                While (Abs(p - j > EPSILON):
                        p=j
                        j = j - (Sqrt(1 - x * x) - cos(j)) / sin(j)
                If (x < -0.99):
                        Return j - π
                Return j
        Else:
                while(Abs(prev - k) > EPSILON)
                        prev = k
                        k = k + (x - sin(k)) / cos(k)
                Return k

arcCos(x) function:
        Return π / 2 - arcSin(x)

arcTan(x) function:
        Return arcCos(1 / Sqrt(x * x + 1))

Log(x) function:
        k = 1.0
        p = 0.0
        While (Abs(p - k) > EPSILON):
                p = k
                k = k + (x - Exp(k)) / Exp(k)
        Return k
```

Pseudocode for mathlib-test.c

```
Define OPTIONS “asctl”

print_arcSin() function:
        Print “ x        arcSin        Library        Difference\n"
        For (i = -1.0; i <= 1.0; i += 0.1):
                Print “%7.4lf%16.8lf%16.8lf%16.10lf\n", i, arcSin(i), asin(i), arcSin(i) - asin(i)
        Return

Print_arcCos() function:
```

```
        Print " x        arcCos        Library        Difference\n"
        For (i = -1.0; i <= 1.0; i += 0.1):
                Print "%7.4lf%16.8lf%16.8lf%16.10lf\n", i, arcCos(i), acos(i), arcCos(i) - acos(i)
        Return

print_arcTan() function:
        Print " x        arcTan        Library        Difference\n"
        for (i = 1.0; i <= 10.0; i += 0.1):
        Print "%7.4lf%16.8lf%16.8lf%16.10lf\n", i, arcTan(i), atan(i), arcTan(i) - atan(i)
        Return

print_Log() function:
        Print " x        Log           Library        Difference\n"
        for (double i = 1.0; i <= 10.0; i += 0.1):
        Print "%7.4lf%16.8lf%16.8lf%16.10lf\n", i, Log(i), log(i), Log(i) - log(i)
        Return

Main (argc, **argv) function:
        opt=0, sin_test=0, cos_test=0, tan_test=0, log_test=0
        While ((opt=getopt(argc, argv, OPTIONS) != -1):
                switch(opt):
                Case 'a':
                        if (!sin_test):
                                sin_test = 1

                        if (!cos_test):
                                cos_test = 1

                        if (!tan_test):
                                tan_test = 1

                        if (!log_test):
                                log_test = 1

                        break
                case 's':
                        if (!sin_test):
                                sin_test = 1
                        break
                case 'c':
                        if (!cos_test):
                                cos_test = 1
                        Break
                Case 't':
```

```
                if (!tan_test):
                        tan_test = 1
                Break
        Case ‘l’
                if (!log_test):
                        log_test = 1
                Break
        Default:
                Print "Program usage ./mathlib-test -[asctl]\n"
                Print "  -a   Runs all tests (arcsin, arccos, arctan, log)\n"
                Print “  -s   Runs arcsin tests\n"
                Print "  -c   Runs arccos tests\n"
                Print "  -t   Runs arctan tests\n"
                Print "  -l   Runs log tests\n"
                Return 1

If (sin_test):
        print_arcSin()

If (cos_test):
        print_arcCos()

if (tan_test):
        print_arcTan()

If (log_test):
        print_Log()

if (!sin_test && !cos_test && !tan_test && !log_test):
        Print "Program usage ./mathlib-test -[asctl]\n"
        Print "  -a   Runs all tests (arcsin, arccos, arctan, log)\n"
        Print “  -s   Runs arcsin tests\n"
        Print "  -c   Runs arccos tests\n"
        Print "  -t   Runs arctan tests\n"
        Print "  -l   Runs log tests\n"
        return 1

Return 0
```

Draft Work

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

$$\sin^{-1}(y) = x$$

$$\sin(\sin^{-1}(y)) = \sin x$$

$$y = \sin x$$

$$y - \sin x = 0$$

$$x_{k+1} = x_k - \frac{y - \sin x_k}{-\cos x_k}$$

$$x_{k+1} = x_k + \frac{y - \sin x_k}{\cos x_k}$$